BERITA ACARA PERBAIKAN KESALAHAN KODE

TUGAS MANDIRI D.1 — MODUL 7: ADVANCED API DESIGN
Praktikum Pemrograman Backend Lanjut (2 SKS)

Program Studi	D4 Teknik Informatika — Fakultas Vokasi, Universitas Airlangga
Mata Kuliah / SKS	Praktikum Pemrograman Backend Lanjut — Semester 5 (2 SKS)
NIM / Mahasiswa	434241021
Repositori / Folder	TUGAS1_434241021/latihan-fiber/MODUL_7

1. Ringkasan Investigasi dan Metodologi Pelacakan

Sesuai ketentuan Aturan Main pada dokumen Panduan Modul 7 Advanced API Design, Bagian B (Langkah Panduan) memuat tepat SEMBILAN (9) kesalahan yang ditanam secara sengaja oleh dosen pengampu. Tiga (3) kesalahan di antaranya ditolak secara langsung oleh compiler Go (compile error), sedangkan enam (6) kesalahan lainnya lolos kompilasi tetapi menghasilkan perilaku sistem yang keliru (runtime/behavior error).

> **Kaidah Acuan Kebenaran (Ground Truth):** Investigasi didasarkan pada 4 jaminan dokumen: (1) Seluruh Bagian A (Landasan Teori) bernilai benar; (2) Seluruh komentar dalam kode menyatakan perilaku yang SEHARUSNYA; (3) Bagian C (Spesifikasi Penerimaan) adalah sasaran mutlak; dan (4) Penyusunan query parameter SQL tidak memuat kesalahan keamanan.

Urutan pelacakan 9 kesalahan dilakukan secara sistematis:
1. Dengarkan Compiler: Menjalankan 'go build ./...' untuk langsung menemukan 3 kesalahan sintaksis/tipe.
2. Bandingkan Komentar dengan Kode: Komentar kode menyatakan maksud sebenarnya; ketidaksesuaian kode membongkar bug logika.
3. Bandingkan Bagian A vs Bagian B: Teori menyatakan spesifikasi status HTTP (422), sorting DESC, dan log error vs warn.
4. Uji terhadap Bagian C: Menjalankan skenario curl untuk menguji validasi, negosiasi konten CSV, dan cursor pagination.
5. Persempit Alur: Membedakan kesalahan di level repository database vs kesalahan di level service / HTTP handler.

2. Tabel Berita Acara Perbaikan (Tugas Mandiri D.1)

Tabel 6 kolom berikut menyajikan rekapitulasi sembilan kesalahan yang ditemukan beserta diagnosis akar masalah, potongan kode perbaikan, dan rujukan bukti pengujian terhadap Bagian C Spesifikasi Penerimaan.

Berkas & Baris	Jenis	Gejala	Akar Masalah	Perbaikan	Bukti (Ref Bagian C)

`middleware/middleware.go`
`Baris 42 & 56`
	Ditolak compiler	Pesan compiler: 'middleware/middleware.go:42:3: declared and not used: status'
		
		Access log mencatat status 200 pada setiap request yang gagal.
			Variabel 'status' sudah dihitung untuk menangkap status AppError, namun pada slice attrs baris 56 kode memanggil c.Response().StatusCode() yang saat itu masih bernilai default (200) karena ErrorHandler belum berjalan. Variabel 'status' menjadi declared and not used.	`Ganti baris 56 pada slice attrs:`
				
				`slog.Int("status", status),`
					Bagian C.8 No 1 & 9:
					• go build ./... sukses.
					• Access log saat GET /users/999999 mencatat status 404 (bukan 200), identik dengan respons client.

`app/model/user.go Baris 28`
`(Dampak: user_rules.go Baris 9, 10, 22)`
	Ditolak compiler	Pesan compiler: 'app/service/user_rules.go:9:21: invalid operation: req.Username != nil (mismatched types string and untyped nil)'
		'user_rules.go:10:41: cannot indirect req.Username'
			Field Username pada PatchUserRequest dideklarasikan bertipe nilai primitif 'string', bukan pointer '*string'. Tipe string tidak dapat dibandingkan dengan nil atau di-dereference. Endpoint PATCH membutuhkan pointer agar dapat membedakan field tidak dikirim (nil) dari field bernilai kosong ("").	`Ubah model.PatchUserRequest baris 28:`
				
				`Username *string` `` `json:"username,omitempty" validate:"omitnil,min=3,max=30,alphanum"` ``
					Bagian C.8 No 1 & 5:
					• go build lolos.
					• PATCH {} -> 400 (tidak ada field diubah).
					• PATCH {"username":""} -> 422 (minimal 3 karakter).
					• PATCH valid -> 200 OK.

`config/app.go`
`Baris 63`
	Ditolak compiler	Pesan compiler: 'config/app.go:63:23: appErr.cause undefined (cannot refer to unexported field cause)'	Field 'cause' pada helper.AppError diawali huruf kecil sehingga berstatus unexported dan tidak dapat diakses dari package config. Selain melanggar enkapsulasi, pemanggilan appErr.cause.Error() akan menyebabkan runtime panic jika cause bernilai nil (misal pada error 4xx).	`Ubah baris 63 config/app.go menggunakan method Error():`
				
				`slog.String("error", appErr.Error())`
					Bagian C.8 No 1, 6, & 7 (9.6):
					• go build lolos.
					• Error database tercatat lengkap di server log via appErr.Error(), sementara client menerima INTERNAL_ERROR aman tanpa kebocoran skema SQL.

`helper/errors.go`
`Baris 62`
	Lolos compiler, salah perilaku	Pelanggaran validasi tag (misal registrasi username/email salah) mengembalikan HTTP status 400 Bad Request, bukan	Fungsi pembangun Validation(fields) menginisialisasi Status dengan fiber.StatusBadRequest (400).	`Ubah Status pada Validation():`
				
				`Status: fiber.StatusUnprocessableE`
					Bagian C.8 No 2 & Bagian 9.3:
					curl -s -o /dev/null -w "%{http_code}" POST /auth/register dengan body

		422.	Sesuai standar arsitektur RESTful modern dan Modul 7 Bagian 9.3 & C.8 No 2, kegagalan validasi semantik entitas wajib berstatus HTTP 422 Unprocessable Content.	`ntity,`	tidak valid menghasilkan status 422 Unprocessable Content.

`config/app.go`
`Baris 57–70`
	Lolos compiler, salah perilaku	Error 404/4xx tercatat di log sebagai ERROR dengan msg 'request_failed'. Sebaliknya error 500 server tercatat WARN dengan msg 'request_rejected'.	Kondisi percabangan log terbalik (appErr.Status < 500). Kode menganggap 4xx sebagai kegagalan sistem kita (Error), dan 5xx sebagai penolakan input client (Warn). Ini membunyikan alarm palsu untuk salah ketik URL dan membungkam kegagalan database sesungguhnya.	`Balik kondisi evaluasi status di ErrorHandler:`
				
				`if appErr.Status >= fiber.StatusInternalServerError {`
				
				`logger.Error("request_failed", ...)`
				`} else {`
				
				`logger.Warn("request_rejected", ...)`
				`}`
					Bagian C.7 & Bagian C.8 No 8:
					• GET /users/999999 mencatat WARN (msg: request_rejected).
					• Database mati/query error mencatat ERROR (msg: request_failed) lengkap dengan SQLSTATE.

`app/service/errors.go`
`Baris 17`
	Lolos compiler, salah perilaku	Saat query database gagal (misal koneksi terputus atau SQL error), API merespons HTTP 200 OK dengan payload kosong/null, bukan 500.	Klausa 'default' pada translateError mengembalikan 'nil' alih-alih helper.Internal(err). Handler mengira query berhasil tanpa error, sehingga mengeksekusi helper.Success dengan entitas kosong (melanggar prinsip arsitektur fail-closed).	`Ubah klausa default pada translateError:`
				
				`default:`
				`  return helper.Internal(err)`
					Bagian C.8 No 6:
					Saat query database sengaja dibuat error (skema rusak), endpoint merespons HTTP 500 INTERNAL_ERROR dan TIDAK PERNAH membalas 200 dengan body kosong.

`helper/validator.go`
`Baris 40`
	Lolos compiler, salah perilaku	1. Password lemah ('password123', 'tanpaangka') lolos validasi.
		2. Password kuat justru ditolak dengan pesan string kosong: 'fields':{'password':''}.
			Fungsi helper passwordStrength mengembalikan string pesan error jika lemah, dan string kosong ("") jika kuat. Ekspresi ditulis '!= ""' sehingga bernilai true (valid) saat password lemah. Saat password kuat bernilai false, messageFor memanggil ulang passwordStrength yang menghasilkan "".	`Ubah logika ekspresi custom validation strongpassword:`
				
				`return passwordStrength(fl.Field().String()) == ""`
					Bagian C.8 No 3 & 4 (Bagian 9.3):
					• 'password123' ditolak ('password terlalu umum').
					• 'tanpaangka' ditolak ('harus memuat huruf dan angka').
					• Password kuat diterima tanpa error string kosong.

`app/repository/user_repository.go`
`Baris 200`
	Lolos compiler, salah perilaku	1. Halaman 1 cursor pagination terurut dari user terlama ke terbaru (urutan menaik).	Query SQL FindAfterCursor memakai 'ORDER BY created_at ASC, id ASC'. Padahal klausul	`Ubah klausul ORDER BY pada FindAfterCursor:`
				
				`"SELECT %s FROM users%s`
					Bagian C.8 No 10, 11, 12 (9.1):
					• Halaman 1 terurut dari user TERBARU ke terlama.

		2. Membuka halaman 2 dengan cursor menghasilkan array kosong [] (data hilang).	WHERE cursor memakai perbandingan 'AND (created_at, id) < ($n-1, $n)' yang menuntut arah menurun (DESC). Akibatnya pencarian data lebih kecil pada daftar menaik gagal menemukan baris.	`ORDER BY created_at DESC, id DESC LIMIT $%d"`	• Halaman 2 memuat user selanjutnya tanpa duplikat. • Halaman terakhir tidak memuat next_cursor dan has_more bernilai false.

helper/negotiate.go Baris 57	**Lolos compiler, salah perilaku**	Request dengan 'Accept: text/csv' membalas status 200 dan header Content-Type text/csv, tetapi file yang diunduh client kosong (0 bytes).	Objek encoding/csv.Writer menggunakan internal buffering pada memory. Fungsi langsung memanggil buffer.String() tanpa mengeksekusi writer.Flush(), sehingga data CSV masih tertahan di buffer internal dan belum masuk ke strings.Builder.	Tambahkan `writer.Flush()` sebelum membaca buffer:
				`writer.Flush()`
				`if err := writer.Error(); err != nil {`
				`  return Internal(err)`
				`}`
				`return c.SendString(buffer.String())`
					Bagian C.8 No 13 & Bagian 9.4: curl -H 'Accept: text/csv' /api/v1/users?limit=3 menghasilkan file CSV berisi baris header dan data utuh: 'id,username,email,role,is_active,created_at' (bukan 0 byte).

3. Rincian Teknis dan Pembuktian Perbaikan Per Butir

Bagian ini menjabarkan secara terperinci setiap kesalahan mulai dari kode awal bermasalah, alasan teknis mengapa kode tersebut salah, potongan perbaikan konkret, dan korelasi langsung dengan kriteria pengujian Bagian C.

Kesalahan 1: Variabel Status Access Log 'Declared and Not Used'

Berkas & Baris	middleware/middleware.go (Baris 42 & 56)
Klasifikasi	**Ditolak compiler (Compiler Error)**

Gejala yang Diamati: Compiler menolak eksekusi 'go build ./...' dengan pesan: middleware/middleware.go:42:3: declared and not used: status
Selain itu, jika baris 42 dinonaktifkan, setiap HTTP request yang menghasilkan error (401, 404, 500) tetap tercatat di access log dengan status 200.

Akar Masalah (Root Cause): Sejak Modul 7, error handler dipusatkan pada ErrorHandler Fiber sehingga handler tidak lagi menulis respons secara langsung. Saat c.Next() selesai, c.Response().StatusCode() belum terisi oleh status error (masih default 200). Kode sudah menghitung variabel 'status' dari AppError, namun pada penyusunan slice attrs baris 56 justru tetap memanggil c.Response().StatusCode(). Akibatnya variabel 'status' tidak terpakai sehingga Go compiler menolak build.

Kode Sebelum Perbaikan:

```
// middleware/middleware.go
status := c.Response().StatusCode()
if err != nil {
    var appErr *helper.AppError
    if errors.As(err, &appErr) {
        status = appErr.Status
    } else {
        status = fiber.StatusInternalServerError
    }
}
attrs := []any{
    slog.String("request_id", requestID),
    slog.String("method", c.Method()),
    slog.String("path", c.Path()),
    slog.Int("status", c.Response().StatusCode()), // <-- SALAH: variabel status tidak
dipakai
    ...
}
```

Kode Sesudah Perbaikan:

```
// middleware/middleware.go (SESUDAH DIPERBAIKI)
attrs := []any{
    slog.String("request_id", requestID),
    slog.String("method", c.Method()),
    slog.String("path", c.Path()),
    slog.Int("status", status), // <-- BENAR: memakai variabel status yang sudah dikoreksi
    slog.Duration("duration", time.Since(start)),
    slog.String("ip", c.IP()),
}
```

> **Bukti Verifikasi & Rujukan Bagian C:** Memenuhi Kriteria Bagian C.8 No 1 & 9:
> 1. 'go build ./...' selesai tanpa warning atau error.

> 2. Request curl ke endpoint tidak ditemukan menghasilkan status log yang akurat:
> {"level":"INFO","msg":"http_request","method":"GET","path":"/api/v1/users/999999","status":404}

Kesalahan 2: Tipe Non-Pointer Username pada PatchUserRequest

Berkas & Baris	app/model/user.go (Baris 28) & app/service/user_rules.go (Baris 9, 10, 22)
Klasifikasi	**Ditolak compiler (Compiler Error)**

Gejala yang Diamati: Compiler menolak kompilasi package service dengan tiga baris error:
app/service/user_rules.go:9:21: invalid operation: req.Username != nil (mismatched types string and untyped nil)
app/service/user_rules.go:10:41: invalid operation: cannot indirect req.Username (variable of type string)
app/service/user_rules.go:22:25: invalid operation: req.Username == nil (mismatched types string and untyped nil)

Akar Masalah (Root Cause): Pada HTTP PATCH (pembaruan parsial), field struct harus bertipe pointer (misal *string) agar membedakan tiga kondisi: (1) field tidak dikirim sama sekali (nil), (2) field dikirim string kosong (""), dan (3) field dikirim dengan nilai baru. Struct PatchUserRequest menulis Username string biasa. Nilai tipe string tidak dapat dibandingkan dengan nil dan tidak dapat di-dereference dengan (*req.Username).

Kode Sebelum Perbaikan:

```
// app/model/user.go
type PatchUserRequest struct {
    Username string  `json:"username,omitempty" validate:"omitnil,min=3,max=30,alphanum"` //
<-- SALAH
    Email    *string `json:"email,omitempty"    validate:"omitnil,email,max=120"`
    IsActive *bool   `json:"is_active,omitempty"`
}
```

Kode Sesudah Perbaikan:

```
// app/model/user.go (SESUDAH DIPERBAIKI)
type PatchUserRequest struct {
    Username *string `json:"username,omitempty" validate:"omitnil,min=3,max=30,alphanum"` //
<-- BENAR: pointer
    Email    *string `json:"email,omitempty"    validate:"omitnil,email,max=120"`
    IsActive *bool   `json:"is_active,omitempty"`
}
```

> **Bukti Verifikasi & Rujukan Bagian C:** Memenuhi Kriteria Bagian C.8 No 1 & 5:
> • Kompilasi berhasil sempurna.
> • curl -X PATCH /users/4 -d '{}' -> 400 'tidak ada field yang diubah'.
> • curl -X PATCH /users/4 -d '{"username":""}' -> 422 'minimal 3 karakter'.
> • curl -X PATCH /users/4 -d '{"username":"valid"}' -> 200 OK.

Kesalahan 3: Pelanggaran Enkapsulasi Field cause pada AppError

Berkas & Baris	config/app.go (Baris 63)
Klasifikasi	**Ditolak compiler (Compiler Error)**

Gejala yang Diamati: Compiler menolak kompilasi package config:
config/app.go:63:23: appErr.cause undefined (cannot refer to unexported field cause)

Akar Masalah (Root Cause): Field 'cause' pada helper.AppError sengaja dideklarasikan lowercase ('cause error') di package helper agar tidak bisa diakses oleh package lain demi mencegah kebocoran detail database. Baris 63 config/app.go mencoba memanggil appErr.cause.Error(). Selain dilarang compiler, pemanggilan method .Error() pada cause yang bernilai nil (pada error 4xx) akan memicu runtime nil pointer dereference (panic).

Kode Sebelum Perbaikan:

```
// config/app.go
logger.Error("request_failed",
    slog.String("request_id", requestID),
    slog.String("path", c.Path()),
    slog.String("code", appErr.Code),
    slog.Int("status", appErr.Status),
    slog.String("error", appErr.cause.Error())) // <-- SALAH: unexported & rawan panic
```

Kode Sesudah Perbaikan:

```
// config/app.go (SESUDAH DIPERBAIKI)
logger.Error("request_failed",
    slog.String("request_id", requestID),
    slog.String("path", c.Path()),
    slog.String("code", appErr.Code),
    slog.Int("status", appErr.Status),
    slog.String("error", appErr.Error())) // <-- BENAR: method Error() sudah aman menangani
cause nil
```

> **Bukti Verifikasi & Rujukan Bagian C:** Memenuhi Kriteria Bagian C.8 No 1, 6, 7 & Bagian 9.6:
> • Proyek lulus kompilasi.
> • Ketika skema database diubah (email -> email_x), log mencatat pesan database lengkap di field 'error', sementara client hanya menerima JSON generik INTERNAL_ERROR dengan request_id pelacak.

Kesalahan 4: Status Code Validasi Menggunakan 400 Alih-Alih 422

Berkas & Baris	helper/errors.go (Baris 62)
Klasifikasi	**Lolos compiler, salah perilaku (Logic/Behavior Error)**

Gejala yang Diamati: Panggilan endpoint yang melanggar validasi tag deklaratif (misal POST /api/v1/auth/register dengan email tidak valid) merespons dengan HTTP status 400 Bad Request.

Akar Masalah (Root Cause): Fungsi helper.Validation() menginisialisasi struct AppError dengan Status: fiber.StatusBadRequest. Dalam arsitektur HTTP API modern (RFC 9110) dan spesifikasi modul (Bagian 9.3 & C.8 No 2), status 400 diperuntukkan bagi sintaks request yang rusak (malformed JSON), sedangkan pelanggaran validasi aturan bisnis atau data payload wajib mengembalikan status 422 Unprocessable Content.

Kode Sebelum Perbaikan:

```
// helper/errors.go
func Validation(fields map[string]string) *AppError {
    return &AppError{
        Status: fiber.StatusBadRequest, // <-- SALAH: seharusnya 422
        Code: CodeValidation,
        Message: "validasi gagal",
        Fields: fields,
    }
}
```

Kode Sesudah Perbaikan:

```
// helper/errors.go (SESUDAH DIPERBAIKI)
func Validation(fields map[string]string) *AppError {
    return &AppError{
        Status: fiber.StatusUnprocessableEntity, // <-- BENAR: status 422 Unprocessable
Content
        Code: CodeValidation,
        Message: "validasi gagal",
        Fields: fields,
    }
}
```

> **Bukti Verifikasi & Rujukan Bagian C:** Memenuhi Kriteria Bagian C.8 No 2 & Bagian 9.3:
> curl -X POST localhost:3000/api/v1/auth/register -d '{"username":"a b!","email":"bukan-email"}'
> Status HTTP yang dikembalikan adalah 422 (Unprocessable Content), dan seluruh error dilaporkan pada objek 'fields'.

Kesalahan 5: Logika Percabangan Tingkat Log (Error vs Warn) Terbalik

Berkas & Baris	config/app.go (Baris 57–70)
Klasifikasi	**Lolos compiler, salah perilaku (Logic/Behavior Error)**

Gejala yang Diamati: Error 404 Not Found atau 401 Unauthorized dicatat sebagai ERROR (msg: request_failed), sedangkan error 500 Internal Server Error dicatat sebagai WARN (msg: request_rejected).

Akar Masalah (Root Cause): Kondisi logika percabangan di newErrorHandler ditulis terbalik: 'if appErr.Status < fiber.StatusInternalServerError'. Status < 500 (keluarga 4xx) adalah kesalahan sisi pemakai API dan seharusnya dicatat sebagai WARN. Status >= 500 adalah kegagalan sistem kita sendiri dan wajib dicatat sebagai ERROR. Logika terbalik membuat monitoring sistem penuh bising saat ada user salah ketik URL, namun diam saat database down.

Kode Sebelum Perbaikan:

```
// config/app.go
if appErr.Status < fiber.StatusInternalServerError { // <-- SALAH: kondisi terbalik
    logger.Error("request_failed", ...)
} else {
    logger.Warn("request_rejected", ...)
}
```

Kode Sesudah Perbaikan:

```
// config/app.go (SESUDAH DIPERBAIKI)
```

```
if appErr.Status >= fiber.StatusInternalServerError { // <-- BENAR: >= 500 adalah ERROR
    logger.Error("request_failed",
        slog.String("request_id", requestID),
        slog.String("path", c.Path()),
        slog.String("code", appErr.Code),
        slog.Int("status", appErr.Status),
        slog.String("error", appErr.Error()))
} else {
    logger.Warn("request_rejected", // <-- BENAR: 4xx adalah WARN
        slog.String("request_id", requestID),
        slog.String("path", c.Path()),
        slog.String("code", appErr.Code),
        slog.Int("status", appErr.Status))
}
```

Bukti Verifikasi & Rujukan Bagian C: Memenuhi Kriteria Bagian C.7 & Bagian C.8 No 8:
• curl /users/999999 -> level: WARN, msg: request_rejected, status: 404.
• Matikan database / picu query error -> level: ERROR, msg: request_failed, status: 500.

Kesalahan 6: translateError Mengembalikan nil pada Kasus Default

Berkas & Baris	app/service/errors.go (Baris 17)
Klasifikasi	**Lolos compiler, salah perilaku (Logic/Behavior Error)**

Gejala yang Diamati: Ketika terjadi kesalahan database yang tidak terduga, endpoint API membalas 200 OK dengan payload data kosong/null, seolah-olah operasi berhasil.

Akar Masalah (Root Cause): Pada fungsi translateError, blok switch tidak memiliki penanganan fail-closed: klausa 'default: return nil'. Ketika database menghasilkan error selain ErrNotFound atau ErrDuplicate, translateError mengembalikan nil (tidak ada error). Service menganggap operasi berhasil dan memanggil helper.Success, menyamarkan kegagalan fatal menjadi sukses palsu.

Kode Sebelum Perbaikan:

```
// app/service/errors.go
func translateError(err error, entity string) error {
    switch {
    case errors.Is(err, repository.ErrNotFound):
        return helper.NotFound(entity + " tidak ditemukan")
    case errors.Is(err, repository.ErrDuplicate):
        return helper.Conflict("username sudah dipakai")
    default:
        return nil // <-- SALAH: menyamarkan error server menjadi sukses
    }
}
```

Kode Sesudah Perbaikan:

```
// app/service/errors.go (SESUDAH DIPERBAIKI)
func translateError(err error, entity string) error {
    switch {
    case errors.Is(err, repository.ErrNotFound):
        return helper.NotFound(entity + " tidak ditemukan")
    case errors.Is(err, repository.ErrDuplicate):
        return helper.Conflict("username sudah dipakai")
    default:
        return helper.Internal(err) // <-- BENAR: fail-closed mengembalikan 500 Internal
Server Error
```

```
    }
}
```

Bukti Verifikasi & Rujukan Bagian C: Memenuhi Kriteria Bagian C.8 No 6:
Kegagalan database selalu memicu response 500 INTERNAL_ERROR dan TIDAK PERNAH menghasilkan 200 dengan body kosong.

Kesalahan 7: Logika Boolean Terbalik pada Validator strongpassword

Berkas & Baris	helper/validator.go (Baris 40)
Klasifikasi	**Lolos compiler, salah perilaku (Logic/Behavior Error)**

Gejala yang Diamati: 1. Password lemah (misal 'password123') lolos registrasi.
2. Password kuat ditolak dengan pesan kesalahan berupa string kosong: 'fields':{'password':''}.

Akar Masalah (Root Cause): Fungsi helper passwordStrength() mengembalikan pesan string alasan penolakan jika password lemah, dan string kosong ("") jika password kuat. Pada registrasi validasi v.RegisterValidation("strongpassword", ...), fungsi ditulis 'return passwordStrength(...) != "". Ini berarti validator mengembalikan TRUE (valid) ketika ada pesan error (lemah), dan mengembalikan FALSE (tidak valid) saat password kuat. Saat validator gagal, messageFor memanggil passwordStrength yang mengembalikan string kosong "".

Kode Sebelum Perbaikan:

```
// helper/validator.go
_ = v.RegisterValidation("strongpassword", func(fl validator.FieldLevel) bool {
    return passwordStrength(fl.Field().String()) != "" // <-- SALAH: terbalik
})
```

Kode Sesudah Perbaikan:

```
// helper/validator.go (SESUDAH DIPERBAIKI)
_ = v.RegisterValidation("strongpassword", func(fl validator.FieldLevel) bool {
    return passwordStrength(fl.Field().String()) == "" // <-- BENAR: valid jika pesan kosong
})
```

Bukti Verifikasi & Rujukan Bagian C: Memenuhi Kriteria Bagian C.8 No 3 & 4:
- Password 'password123' ditolak: "fields":{"password":"password terlalu umum"}.
- Password 'tanpaangka' ditolak: "fields":{"password":"harus memuat huruf dan angka"}.
- Password kuat diterima dan pesan validasi tidak pernah berupa string kosong.

Kesalahan 8: Arah Sorting Query Cursor Pagination Terbalik (ASC Alih-Alih DESC)

Berkas & Baris	app/repository/user_repository.go (Baris 200)
Klasifikasi	**Lolos compiler, salah perilaku (Logic/Behavior Error)**

Gejala yang Diamati: 1. Halaman 1 terurut dari user tertua ke terbaru.
2. Mengambil halaman 2 dengan cursor menghasilkan daftar kosong [].

Akar Masalah (Root Cause): Query SQL FindAfterCursor ditulis dengan 'ORDER BY created_at ASC, id ASC'. Namun klausul pembanding cursor WHERE menggunakan operator lebih kecil: 'WHERE (created_at, id) < ($n-1, $n)'. Dalam urutan menaik (ASC), data berikutnya memiliki nilai yang lebih besar, sehingga perbandingan operator '<' tidak pernah terpenuhi dan mengembalikan 0 baris. Selain itu, spesifikasi Modul 7 Bagian C.8 No 10 mewajibkan data terurut dari yang terbaru (DESC).

Kode Sebelum Perbaikan:

```
// app/repository/user_repository.go
query := fmt.Sprintf(
    "SELECT %s FROM users%s ORDER BY created_at ASC, id ASC LIMIT $%d", // <-- SALAH: ASC
    userColumns, where, len(args))
```

Kode Sesudah Perbaikan:

```
// app/repository/user_repository.go (SESUDAH DIPERBAIKI)
query := fmt.Sprintf(
    "SELECT %s FROM users%s ORDER BY created_at DESC, id DESC LIMIT $%d", // <-- BENAR: DESC
    userColumns, where, len(args))
```

Bukti Verifikasi & Rujukan Bagian C: Memenuhi Kriteria Bagian C.8 No 10, 11, 12 & Bagian 9.1:
• Halaman 1 menampilkan user terbaru secara menurun.
• Halaman 2 memuat user selanjutnya tanpa ada data yang duplikat atau terlewat.
• Halaman terakhir menyajikan has_more: false tanpa next_cursor.

Kesalahan 9: Kurangnya writer.Flush() pada WriteUsersCSV

Berkas & Baris	helper/negotiate.go (Baris 57)
Klasifikasi	**Lolos compiler, salah perilaku (Logic/Behavior Error)**

Gejala yang Diamati: Request dengan header 'Accept: text/csv' merespons status 200 OK dan header attachment CSV, tetapi isi file yang diterima client kosong (0 bytes).

Akar Masalah (Root Cause): Objek csv.NewWriter(&buffer) dari paket standar Go encoding/csv menggunakan internal memory buffer untuk efisiensi penulisan data. Data yang ditulis dengan writer.Write() belum dituangkan ke target strings.Builder sebelum method writer.Flush() dipanggil. Karena kode langsung memanggil buffer.String() tanpa Flush(), seluruh baris data CSV tertahan di memori internal dan terbuang.

Kode Sebelum Perbaikan:

```
// helper/negotiate.go
for _, u := range users {
    row := []string{...}
    if err := writer.Write(row); err != nil { return Internal(err) }
}
// <-- SALAH: writer.Flush() tidak pernah dipanggil
if err := writer.Error(); err != nil {
    return Internal(err)
}
return c.SendString(buffer.String())
```

Kode Sesudah Perbaikan:

```
// helper/negotiate.go (SESUDAH DIPERBAIKI)
for _, u := range users {
    row := []string{...}
    if err := writer.Write(row); err != nil { return Internal(err) }
}
writer.Flush() // <-- BENAR: menuangkan seluruh sisa buffer ke strings.Builder
if err := writer.Error(); err != nil {
    return Internal(err)
}
return c.SendString(buffer.String())
```

Bukti Verifikasi & Rujukan Bagian C: Memenuhi Kriteria Bagian C.8 No 13 & Bagian 9.4: curl -H 'Accept: text/csv' 'localhost:3000/api/v1/users?limit=3' menghasilkan file CSV lengkap berisi header dan 3 baris data user riil (bukan berkas kosong).

4. Rekapitulasi Daftar Periksa Ringkas (Spesifikasi Bagian C.8)

Setelah kesembilan perbaikan di atas diterapkan pada codebase, seluruh 14 butir spesifikasi penerimaan pada Bagian C.8 terpenuhi secara sempurna sebagai berikut:

No	Perilaku yang Harus Benar (Spesifikasi Modul)	Status
1	go build ./... berjalan tanpa satu pun pesan kesalahan	**TERPENUHI**
2	Aturan tag dilanggar -> status 422, bukan 400	**TERPENUHI**
3	Pesan validasi tidak pernah berupa string kosong	**TERPENUHI**
4	Password kuat DITERIMA; password lemah, terlalu pendek, atau terlalu umum DITOLAK	**TERPENUHI**
5	PATCH tanpa field apa pun -> 400; PATCH dengan {"username":""} -> 422; PATCH sah -> 200	**TERPENUHI**
6	Kegagalan database -> 500 INTERNAL_ERROR, TIDAK PERNAH 200 dengan body kosong	**TERPENUHI**
7	Pesan asli database tidak pernah muncul pada response, hanya pada log	**TERPENUHI**
8	4xx tercatat WARN; 5xx tercatat ERROR	**TERPENUHI**
9	Status pada access log sama dengan status yang diterima client	**TERPENUHI**
10	Halaman 1 terurut dari yang TERBARU ke yang terlama	**TERPENUHI**
11	Halaman 2 tidak memuat satu pun baris yang sudah muncul di halaman 1	**TERPENUHI**
12	Halaman terakhir tidak menyertakan next_cursor dan has_more bernilai false	**TERPENUHI**
13	Accept: text/csv menghasilkan berkas BERISI data, bukan berkas kosong	**TERPENUHI**
14	Accept: application/xml -> 406; Accept: */* -> 200 JSON	**TERPENUHI**